

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

#LOAD THE DATASET FOR THE IPL MATCHES

df=pd.read_csv('E:\\Data_Science\\EDA\\IPL_Dataset.csv')

df
```

Out[2]:

	Jersey No	Player	Matches	Inns	Not Out	Runs	Highest Score	Avg	Balls faced	Strike rate	100
0	1	KL Rahul	14	14	2	670	132*	55.83	518	129.34	1
1	2	Shikhar Dhawan	17	17	3	618	106*	44.14	427	144.73	2
2	3	David Warner	16	16	2	548	85*	39.14	407	134.64	0
3	4	Shreyas Iyer	17	17	2	519	88*	34.60	421	123.27	0
4	5	Ishan Kishan	14	13	4	516	99	57.33	354	145.76	0
...	...	...	...	...	...	...	...	...	...	...	...
128	129	Khaleel Ahmed	7	1	0	0	0*	0.00	2	0.00	0
129	130	Arshdeep Singh	8	1	0	0	0*	0.00	3	0.00	0
130	131	Daniel Sams	3	1	0	0	0*	0.00	2	0.00	0
131	132	Shreevats Goswami	2	2	0	0	0*	0.00	4	0.00	0
132	133	Trent Boult	15	1	0	0	0*	0.00	1	0.00	0

133 rows × 14 columns



```
In [6]: # Data cleaning with the columns names and mapping values
df.columns = df.columns.str.strip()
df.rename(columns={df.columns[10]: "100s", df.columns[11]: "50s"}, inplace=True)
print(df.columns)
```

```
Index(['Jersey No', 'Player', 'Matches', 'Inns', 'Not Out', 'Runs',
      'Highest Score', 'Avg', 'Balls faced', 'Strike rate', '100s', '50s',
      '4s', '6s'],
      dtype='object')
```

```
In [5]: #Q1. What is the maximum number of matches played by an individual player in a s
max_matches = df["Matches"].max()
ind_play_max_matches = df[df["Matches"] == max_matches][["Player", "Matches"]]
print(ind_play_max_matches)
```

	Player	Matches
1	Shikhar Dhawan	17
3	Shreyas Iyer	17
19	Marcus Stoinis	17
67	Kagiso Rabada	17

```
In [9]: #Q2. Top 2 players with maximum Average who have scored atleast 2 half centuries

df["50s"] = pd.to_numeric(df["50s"], errors = 'coerce').fillna(0).astype(int).in

df["Avg"] = pd.to_numeric(df["Avg"], errors = 'coerce')
print("Unique values in Avg column after conversion:", df["Avg"].unique())
qualified_players = df[df["50s"] >= 2].copy()

if not qualified_players.empty:
    top_avg_player = qualified_players.nlargest(2, "Avg")[["Player", "Avg"]]
    print(top_avg_player)
```

Unique values in Avg column after conversion: [55.83 44.14 39.14 34.6 57.33
35.92 40. 31.53 42.36 45.4
40.81 33.84 32.69 38.54 41.8 28.84 39.88 35.3 25.14 31.36
31.18 27.66 32.8 45.28 25.91 29.9 41.14 40.71 35.12 22.33
53.6 42.5 46.4 23. 17.53 71.33 51. 25. 16.33 23.25
23.12 14.08 20.85 14.77 21.66 18.42 13.44 13. 14.62 18.83
14.12 18.5 22.2 18.16 15.42 20.6 101. 24.25 12.28 83.
19.5 0. 14.2 33. 20.66 19.66 14. 42. 13.33 9.25
12.33 8.75 8.5 16.5 10.66 16. 9.66 27. 7.33 21.
10.5 9.5 9. 17. 8. 7.5 5. 15. 4. 6.
12. 11. 3.5 7. 2. 1.]

	Player	Avg
57	Deepak Hooda	101.0
60	Tom Curran	83.0

```
In [10]: #Q3. Create 2 new columns based on Player name. First column will have first nam
df["Player"] = df["Player"].astype(str)
df[["First_Name", "Last_Name"]] = df["Player"].str.split(n=1, expand=True)

print(df[["Player", "First_Name", "Last_Name"]].head())
```

	Player	First_Name	Last_Name
0	KL Rahul	KL	Rahul
1	Shikhar Dhawan	Shikhar	Dhawan
2	David Warner	David	Warner
3	Shreyas Iyer	Shreyas	Iyer
4	Ishan Kishan	Ishan	Kishan

```
In [14]: #Q4. Create a new column (Cleaned_Highest_score) based on Highest score variable
df["Cleaned_Highest_Score"] = df["Highest Score"].astype(str).str.replace('*', ' ')
df["Cleaned_Highest_Score"] = pd.to_numeric(df["Cleaned_Highest_Score"], errors
print(df[["Player", "Cleaned_Highest_Score"]].head())
```

	Player	Cleaned_Highest_Score
0	KL Rahul	132
1	Shikhar Dhawan	106
2	David Warner	85
3	Shreyas Iyer	88
4	Ishan Kishan	99

In [16]: *#Q5. Print the total number of centuries scored in the entire season.*

```
df["Runs"] = pd.to_numeric(df["Runs"], errors="coerce").fillna(0).astype(int)
total_centuries_count = (df['Runs'] >=100).sum()
print(f"The total number of centuries scored in the entire season is {total_centuries_count}")
```

The total number of centuries scored in the entire season is 58.

In [17]: *#Q6. Print all the player names whose strike rate is less than the average strike rate.*

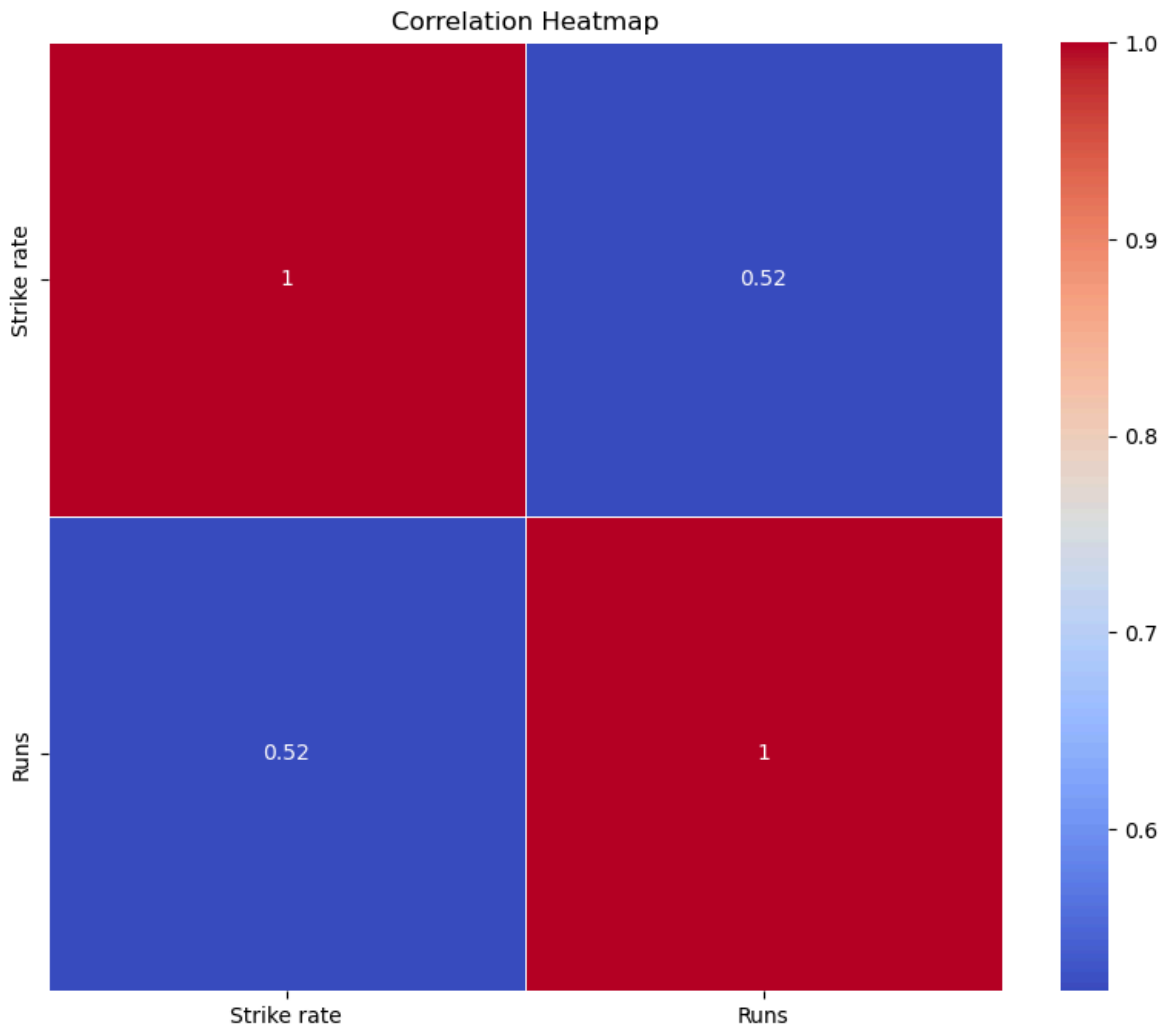
```
avg_strike_rate = df["Strike rate"].mean()
low_strike_rate_players = df[df["Strike rate"] < avg_strike_rate][["Player", "Strike rate"]]
print(low_strike_rate_players)
```

	Player	Strike rate
51	Ajinkya Rahane	105.60
55	Glenn Maxwell	101.88
58	Vijay Shankar	101.04
61	Josh Philippe	101.29
62	Gurkeerat Singh	88.75
65	Kedar Jadhav	93.93
70	Yashasvi Jaiswal	90.90
71	Shreyas Gopal	94.87
77	Murali Vijay	74.41
79	Chris Jordan	93.54
80	Navdeep Saini	100.00
82	Kamlesh Nagarkoti	70.96
84	Harshal Patel	87.50
85	Jimmy Neesham	105.55
86	Tom Banton	90.00
89	Prabhsimran Singh	100.00
92	Kuldeep Yadav	61.90
94	Moeen Ali	75.00
95	Sandeep Sharma	80.00
96	Shardul Thakur	57.14
98	Rinku Singh	100.00
99	Shivam Mavi	71.42
100	Varun Chakaravarthi	66.66
101	Jaydev Unadkat	69.23
102	Ankit Rajpoot	90.00
104	Shahbaz Nadeem	87.50
105	Pravin Dubey	53.84
107	Deepak Chahar	58.33
108	Ravi Bishnoi	58.33
109	Andrew Tye	100.00
111	Kartik Tyagi	66.66
112	Murugan Ashwin	100.00
114	T Natarajan	60.00
115	Prasidh Krishna	50.00
116	Rahul Chahar	50.00
117	Mohammad Shami	66.66
118	Nikhil Naik	33.33
119	Mujeeb Ur Rahman	33.33
120	Dale Steyn	50.00
121	Varun Aaron	10.00
122	Shahbaz Ahmed	100.00
123	Yuzvendra Chahal	33.33
124	Mitchell Marsh	0.00
125	Umesh Yadav	0.00
126	Bhuvneshwar Kumar	0.00
127	Sheldon Cottrell	0.00
128	Khaleel Ahmed	0.00
129	Arshdeep Singh	0.00
130	Daniel Sams	0.00
131	Shreevats Goswami	0.00
132	Trent Boult	0.00

```
In [18]: #Q7. Please check the correlation between the features and create a heat map.
correlation_matrix = df[['Strike rate', 'Runs']].corr()

# Create a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=0.5)
```

```
plt.title('Correlation Heatmap')
plt.show()
```



```
In [19]: #Q8. Check the list of players who has an average greater than 50 as well strike
# Filter players with average greater than 50 and strike rate above 120
filtered_players = df[(df['Avg'] > 50) & (df['Strike rate'] > 120)]

# Print the player name, average and strike rate
for index, row in filtered_players.iterrows():
    print(f"Player Name: {row['Player']}, Average: {row['Avg']}, Strike Rate: {r
```

```
Player Name: KL Rahul, Average: 55.83, Strike Rate: 129.34
Player Name: Ishan Kishan, Average: 57.33, Strike Rate: 145.76
Player Name: Kieron Pollard, Average: 53.6, Strike Rate: 191.42
Player Name: Wriddhiman Saha, Average: 71.33, Strike Rate: 139.86
Player Name: Ruturaj Gaikwad, Average: 51.0, Strike Rate: 120.71
Player Name: Deepak Hooda, Average: 101.0, Strike Rate: 142.25
Player Name: Tom Curran, Average: 83.0, Strike Rate: 133.87
```

```
In [20]: #Q9. Please check the list of players who has an average greater than 40 and bal
# Filter players with average greater than 40 and balls faced above 100
filtered_players = df[(df['Avg'] > 40) & (df['Balls faced'] > 100)]

# Print the player name, average and balls faced
for index, row in filtered_players.iterrows():
    print(f"Player Name: {row['Player']}, Average: {row['Avg']}, Balls Faced: {r
```

Player Name: KL Rahul, Average: 55.83, Balls Faced: 518
 Player Name: Shikhar Dhawan, Average: 44.14, Balls Faced: 427
 Player Name: Ishan Kishan, Average: 57.33, Balls Faced: 354
 Player Name: Virat Kohli, Average: 42.36, Balls Faced: 384
 Player Name: ABD Villiers, Average: 45.4, Balls Faced: 286
 Player Name: Faf Duplessis, Average: 40.81, Balls Faced: 319
 Player Name: Eoin Morgan, Average: 41.8, Balls Faced: 302
 Player Name: Kane Williamson, Average: 45.28, Balls Faced: 237
 Player Name: Chris Gayle, Average: 41.14, Balls Faced: 210
 Player Name: Ben Stokes, Average: 40.71, Balls Faced: 200
 Player Name: Kieron Pollard, Average: 53.6, Balls Faced: 140
 Player Name: Rahul Tewatia, Average: 42.5, Balls Faced: 183
 Player Name: Ravindra Jadeja, Average: 46.4, Balls Faced: 135
 Player Name: Wriddhiman Saha, Average: 71.33, Balls Faced: 153
 Player Name: Ruturaj Gaikwad, Average: 51.0, Balls Faced: 169

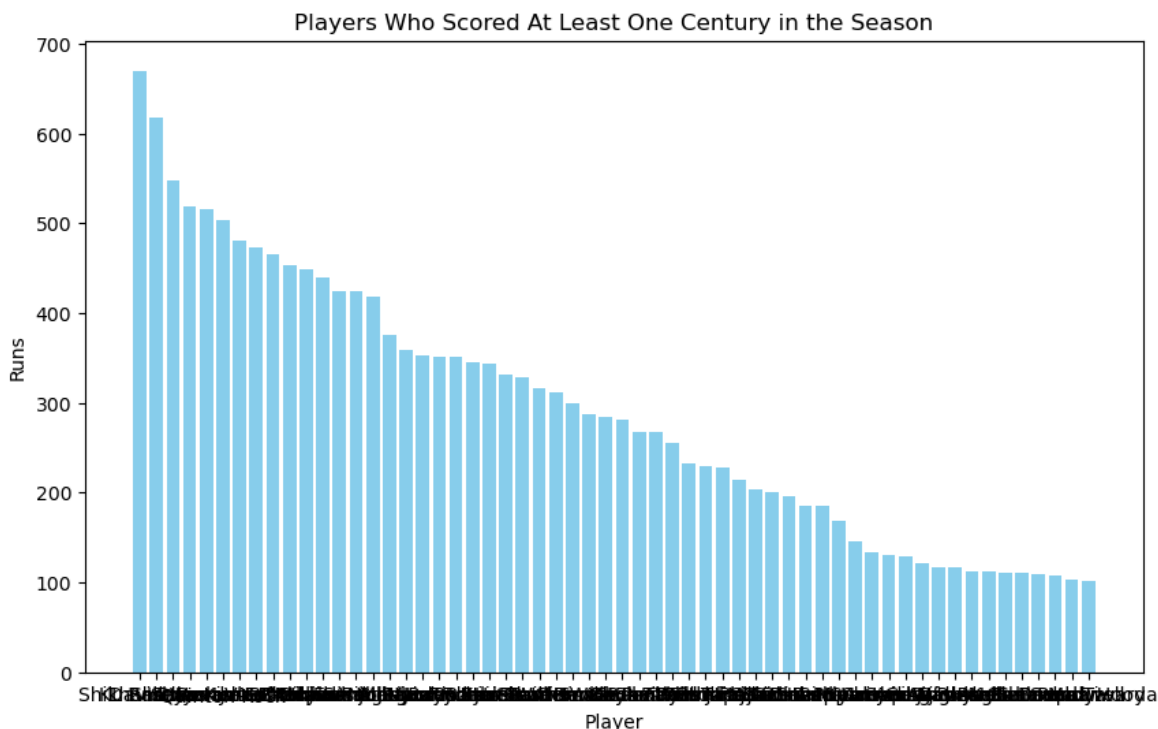
In [21]: *#Q10. Players who scored atleast one century in this season. Create visualization*

```

# Filter players who scored at Least one century (score >= 100)
century_scorers = df[df['Runs'] >= 100]

# Create a bar plot for players who scored at Least one century
plt.figure(figsize=(10, 6))
plt.bar(century_scorers['Player'], century_scorers['Runs'], color='skyblue')
plt.xlabel('Player')
plt.ylabel('Runs')
plt.title('Players Who Scored At Least One Century in the Season')
plt.show()

```



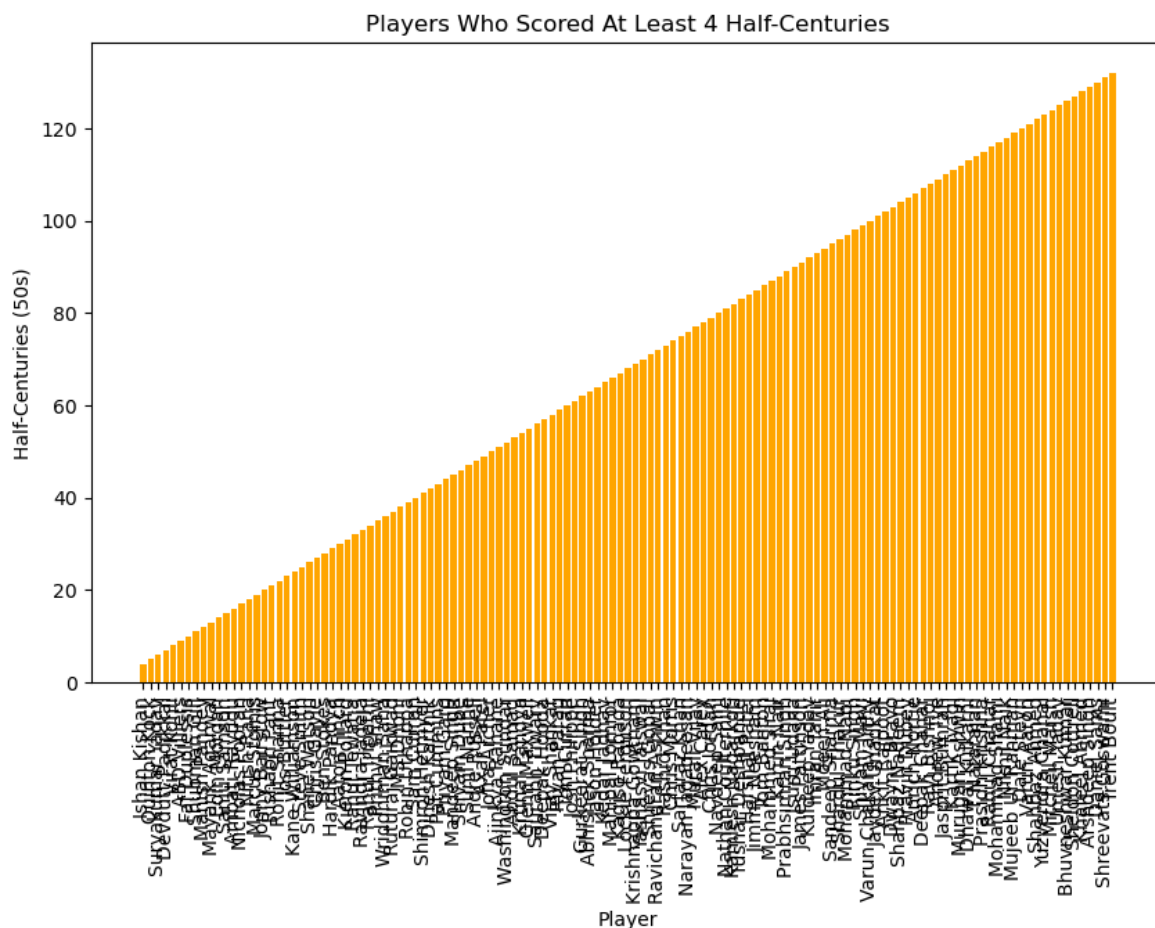
In [24]: `print(df.columns[df.columns.duplicated()])`

Index([], dtype='object')

In [25]: `df = df.loc[:, ~df.columns.duplicated()]`

In [36]: *##Q11. Players who scored atleast 4 half centuries in this season.
Ensure "50s" column is numeric*

```
df["50s"] = pd.to_numeric(df["50s"], errors="coerce").fillna(0).astype(int)
half_century_players = df[df["50s"] >= 4]
plt.figure(figsize=(10, 6))
plt.bar(half_century_players["Player"], half_century_players["50s"], color="orange")
plt.xlabel("Player")
plt.ylabel("Half-Centuries (50s)")
plt.title("Players Who Scored At Least 4 Half-Centuries")
plt.xticks(rotation=90)
plt.show()
print("Players who scored at least 4 half-centuries in the season:")
print(half_century_players[["Player", "50s"]])
```



Players who scored at least 4 half-centuries in the season:

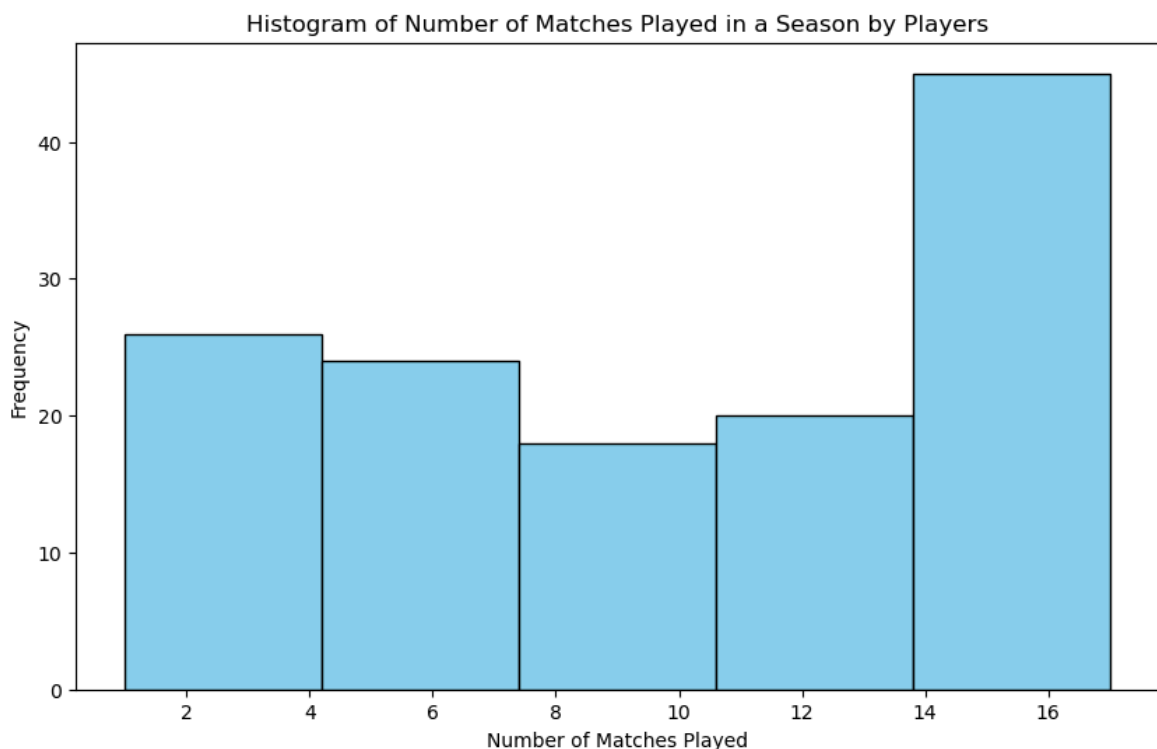
	Player	50s
4	Ishan Kishan	4
5	Quinton Kock	5
6	Suryakumar Yadav	6
7	Devdutt Padikkal	7
8	Virat Kohli	8
..	...	...
128	Khaleel Ahmed	128
129	Arshdeep Singh	129
130	Daniel Sams	130
131	Shreevats Goswami	131
132	Trent Boult	132

[129 rows x 2 columns]

In [39]: #Q12. Check the List of players who hit more than 45 boundaries and more than 10
 boundary_hitters = df[(df["4s"] > 45) & (df["6s"] > 10)][["Player", "4s", "6s"]]
 print(boundary_hitters)

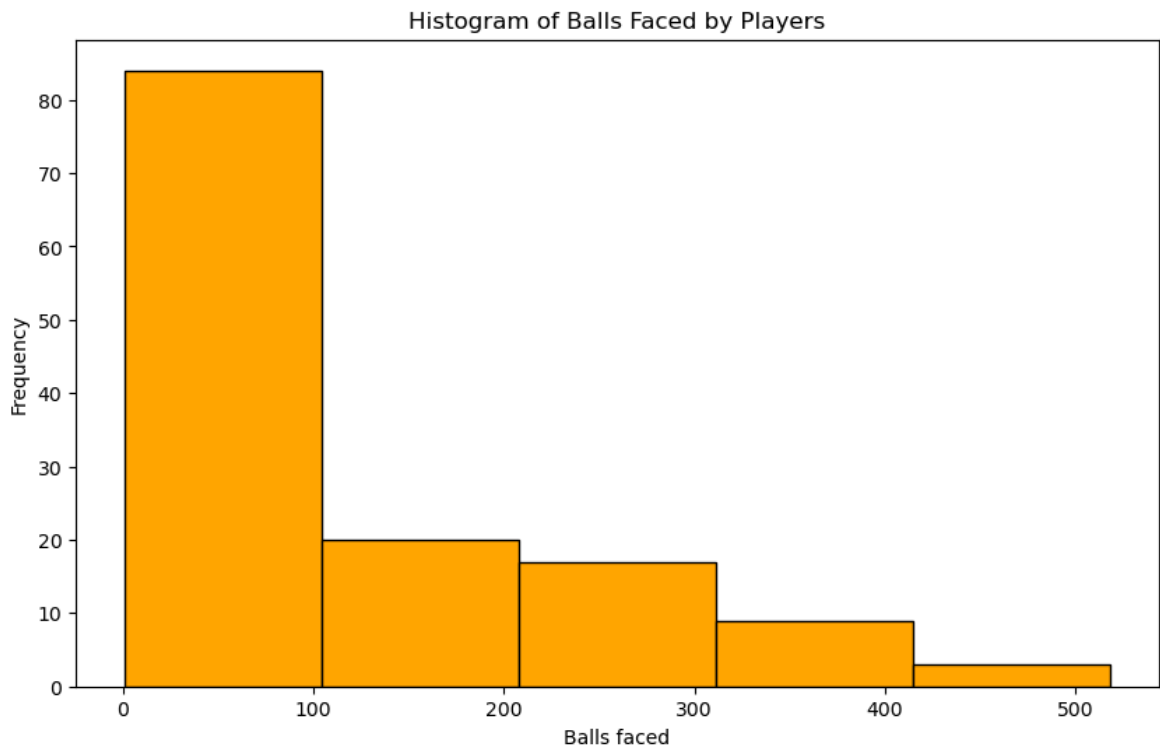
	Player	4s	6s
0	KL Rahul	58	23
1	Shikhar Dhawan	67	12
2	David Warner	52	14
5	Quinton Kock	46	22
6	Suryakumar Yadav	61	11

```
In [40]: #Q13. Plot a histogram of number of matches played in a season by players.
# Plot a histogram of number of matches played in a season by players
plt.figure(figsize=(10, 6))
plt.hist(df['Matches'], bins=5, color='skyblue', edgecolor='black')
plt.xlabel('Number of Matches Played')
plt.ylabel('Frequency')
plt.title('Histogram of Number of Matches Played in a Season by Players')
plt.show()
```



```
In [41]: #Q14. Plot the histogram of balls faced by players.

# Plot a histogram of balls faced by players
plt.figure(figsize=(10, 6))
plt.hist(df['Balls faced'], bins=5, color='orange', edgecolor='black')
plt.xlabel('Balls faced')
plt.ylabel('Frequency')
plt.title('Histogram of Balls Faced by Players')
plt.show()
```



```
In [42]: #Q15. Top 10 players with most runs in a season.
# Sort the DataFrame by Runs in descending order and get the top 10 players
top_10_players = df.sort_values(by='Runs', ascending=False).head(10)

# Print the player name and runs
for index, row in top_10_players.iterrows():
    print(f"Player Name: {row['Player']}, Runs: {row['Runs']}")
```

```
Player Name: KL Rahul, Runs: 670
Player Name: Shikhar Dhawan, Runs: 618
Player Name: David Warner, Runs: 548
Player Name: Shreyas Iyer, Runs: 519
Player Name: Ishan Kishan, Runs: 516
Player Name: Quinton Kock, Runs: 503
Player Name: Suryakumar Yadav, Runs: 480
Player Name: Devdutt Padikkal, Runs: 473
Player Name: Virat Kohli, Runs: 466
Player Name: ABD Villiers, Runs: 454
```

```
In [52]: #Q16. Print the players who played the match but didn't get the batting.
# Filter players who played the match but didn't get to bat
players_didnt_bat = df[(df['Matches'] == True) & (df['Inns'] == False)]

# Print the player names
for index, row in players_didnt_bat.iterrows():
    print(f"Player Name: {row['Player']}")
```

```
In [45]: #Q17. Create a new column to show the percentage of total runs scored in 4s and
df["Boundary_Percentage"] = ((df["4s"] * 4 + df["6s"] * 6) / df["Runs"]) * 100
top_boundary_players = df.nlargest(5, "Boundary_Percentage")[['Player', 'Boundary_Percentage']]
print(top_boundary_players)
```

	Player	Boundary_Percentage
109	Andrew Tye	100.000000
48	Andre Russell	76.923077
74	Chris Morris	76.470588
29	Hardik Pandya	73.309609
47	Sunil Narine	72.727273

```
In [53]: #Q18. Print the players with top 5 Not out percentages (Not Out percentage can b
# Calculate Not Out percentage
df['Not_Out_Percentage'] = df['Not Out'] / df['Inns'] * 100

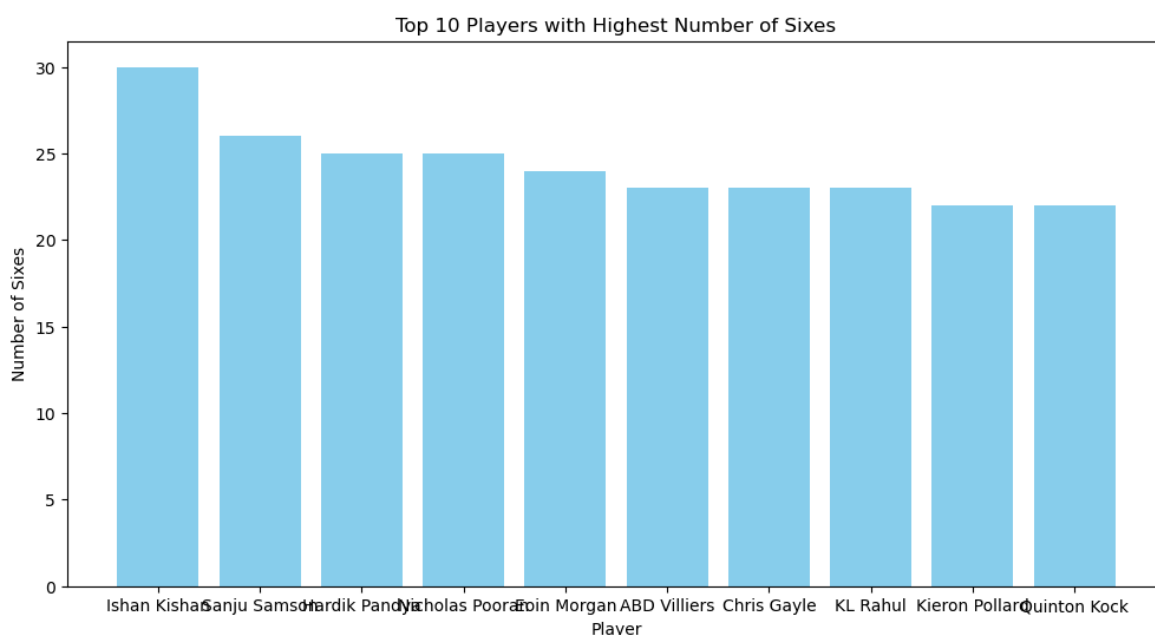
# Sort the DataFrame by Not Out percentage in descending order and get the top 5
top_5_not_out_percentage = df.sort_values(by='Not_Out_Percentage', ascending=False)

# Print the player name and Not Out percentage
for index, row in top_5_not_out_percentage.iterrows():
    print(f"Player Name: {row['Player']}, Not Out Percentage: {row['Not_Out_Perc
```

Player Name: Shahbaz Ahmed, Not Out Percentage: 100.00%
 Player Name: Mohammad Nabi, Not Out Percentage: 100.00%
 Player Name: T Natarajan, Not Out Percentage: 100.00%
 Player Name: Rahul Chahar, Not Out Percentage: 100.00%
 Player Name: Dhawal Kulkarni, Not Out Percentage: 100.00%

```
In [54]: #Q19. Create visualization of top 10 players with highest number of sixes.
# Sort the DataFrame by Sixes in descending order and get the top 10 players
top_10_sixes = df.sort_values(by='6s', ascending=False).head(10)

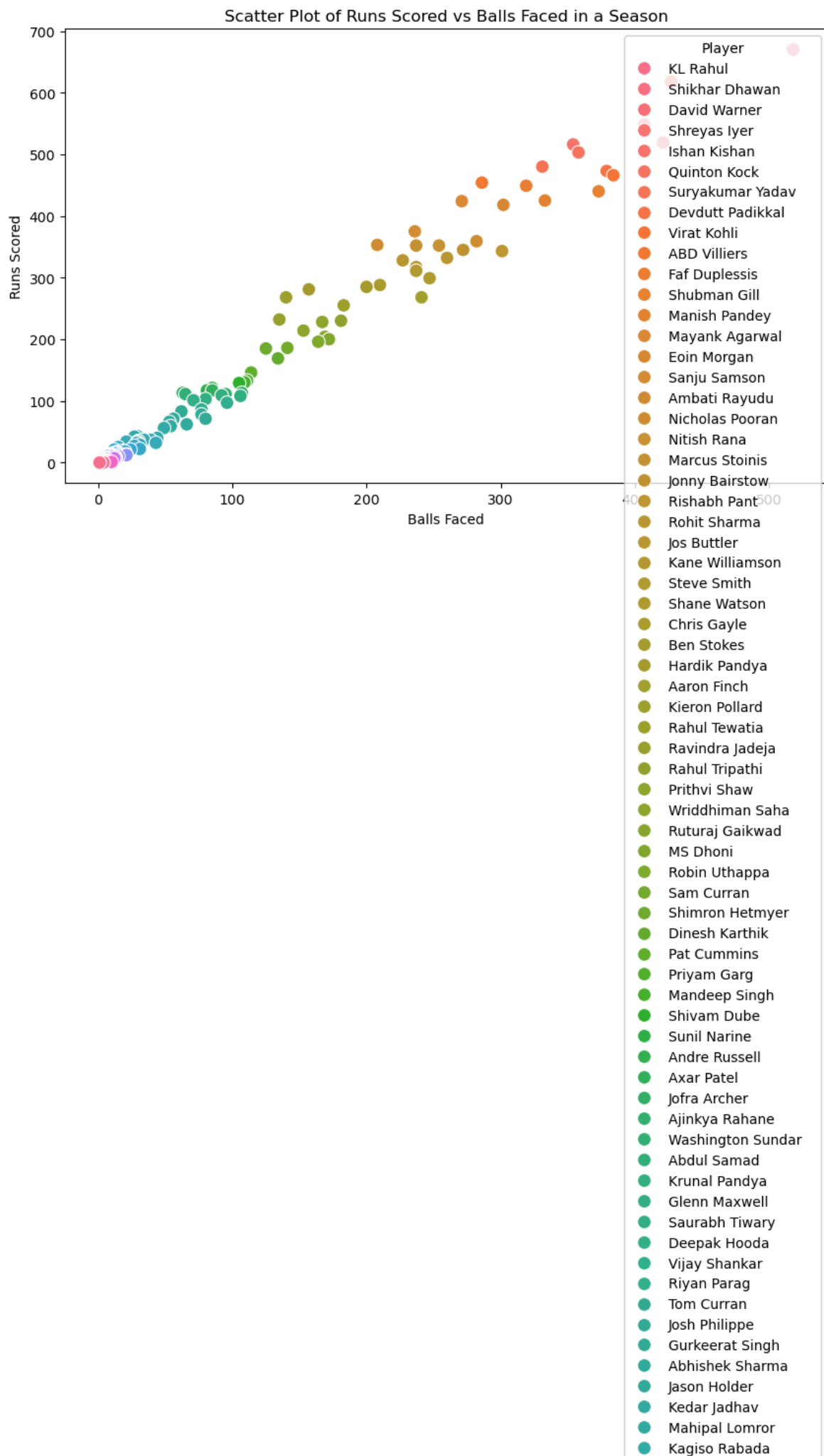
# Create a bar plot for the top 10 players with the highest number of sixes
plt.figure(figsize=(12, 6))
plt.bar(top_10_sixes['Player'], top_10_sixes['6s'], color='skyblue')
plt.xlabel('Player')
plt.ylabel('Number of Sixes')
plt.title('Top 10 Players with Highest Number of Sixes')
plt.show()
```



```
In [55]: #Q20. Scatter plot of runs scored by a player v/s balls faced in a season. Then
# Create a scatter plot of runs scored by a player vs balls faced in a season
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Balls faced', y='Runs', data=df, hue='Player', s=100)
```

```
plt.xlabel('Balls Faced')
plt.ylabel('Runs Scored')
plt.title('Scatter Plot of Runs Scored vs Balls Faced in a Season')
plt.show()

# Calculate the correlation between runs scored and balls faced
correlation = df['Runs'].corr(df['Balls faced'])
print(f"The correlation between runs scored and balls faced is {correlation:.2f}")
```



- Lockie Ferguson
- Krishnappa Gowtham
- Yashasvi Jaiswal
- Shreyas Gopal
- Ravichandran Ashwin
- Rashid Khan
- Chris Morris
- Sarfaraz Khan
- Narayan Jagadeesan
- Murali Vijay
- Alex Carey
- Chris Jordan
- Navdeep Saini
- Nathan Coulternile
- Kamlesh Nagarkoti
- Tushar Deshpande
- Harshal Patel
- Jimmy Neesham
- Tom Banton
- Mohammed Siraj
- Karun Nair
- Prabhsimran Singh
- Isuru Udana
- James Pattinson
- Kuldeep Yadav
- Imran Tahir
- Moeen Ali
- Sandeep Sharma
- Shardul Thakur
- Mohammad Nabi
- Rinku Singh
- Shivam Mavi
- Varun Chakaravarthy
- Jaydev Unadkat
- Ankit Rajpoot
- Dwayne Bravo
- Shahbaz Nadeem
- Pravin Dubey
- Anrich Nortje
- Deepak Chahar
- Ravi Bishnoi
- Andrew Tye
- Jasprit Bumrah
- Kartik Tyagi
- Murugan Ashwin
- Dhawal Kulkarni
- T Natarajan
- Prasidh Krishna
- Rahul Chahar
- Mohammad Shami
- Nikhil Naik
- Mujeeb Ur Rahman
- Dale Steyn
- Varun Aaron
- Shahbaz Ahmed
- Yuzvendra Chahal
- Mitchell Marsh
- Umesh Yadav
- Bhuvneshwar Kumar
- Sheldon Cottrell
- Khaleel Ahmed
- Arshdeep Singh
- Daniel Sams
- Shreevats Goswami
- Trent Boult

The correlation between runs scored and balls faced is 0.99.

In []: